

CODE REVIEW REPORT

NIPP — Property NI Multi-Tenant Portal

Generated: July 24, 2026

Executive Summary

This report presents a comprehensive code review of the **NIPP** (Property NI) codebase — a multi-tenant property management portal built on Next.js 15, Better Auth, Prisma/PostgreSQL with Row Level Security, and Redis caching. The review covers three dimensions: **codebase overview**, **performance analysis**, and **security audit**.

The codebase demonstrates strong architectural thinking with defense-in-depth multi-tenant isolation, role-based access control, and CSP nonces. However, several critical and high-severity issues were identified across all three dimensions that must be addressed before production deployment.

Architecture	Strong
	Needs Improvement
Security	Moderate
	Good
Code Quality	Good

1. Codebase Overview

1.1 Technology Stack

Layer	Technology	Version
Runtime	Node.js	22 LTS (pinned)
Framework	Next.js	15.x (App Router)
UI Library	React	19.x
Authentication	Better Auth	1.6.x (with Organization plugin)
ORM	Prisma	6.x
Database	PostgreSQL	16+ (via PgBouncer)
Caching	Redis (ioredis)	7+ (graceful degradation)
Styling	Tailwind CSS	v4.x
Validation	Zod	4.x
Logging	Pino	9.x
Testing	Vitest	4.1.x (~10K LOC, ~35 test files)

1.2 Architecture Overview

The app follows a **defense-in-depth multi-tenant architecture** with three layers:

- **Layer 1: Edge Middleware** — CSP nonce generation, session cookie validation, tenant context setup via AsyncLocalStorage.
- **Layer 2: API Routes** — Node.js runtime routes with `withAuth()/withSuperAdmin()` wrappers, service layer business logic.
- **Layer 3: Database** — Two-layer tenant isolation: Prisma `$extends` query interception + PostgreSQL Row Level Security (RLS).

Super Admins use a separate unscoped Prisma client (`globalDb`) to query across all organizations. Regular users are scoped via `tenantDb` which auto-injects `organizationId` into every query.

1.3 Feature Surface

- **Authentication** Email/password (sign-ups disabled), Google OAuth, HTTP-only session cookies with Edge-safe cookie cache
- **Super Admin Dashboard** Organization CRUD (list/create/view/status-change), permission catalog, audit log viewer, system health monitoring
- **RBAC** 9 bootstrapped roles (`super_admin`, `admin`, `owner`, `manager`, `agent`, `maintenance`, `tenant`, `accountant`, `contractor`), role-to-permission mapping, member-to-role assignment
- **Notifications** SSE streaming endpoint + React Query cache injection with 60s polling fallback

- **Multi-tenant Org Management** Organization settings, user profile management

1.4 Directory Structure

Directory	Purpose
app/	Next.js pages + API routes (16 endpoints)
components/	Admin UI, auth guards, org switcher
features/	Feature-sliced modules (auth, notifications, organization, permissions, user)
hooks/	Custom React hooks (usePermission)
lib/	Core logic: auth, authz, tenant isolation, caching, schemas, crypto
prisma/	Schema + 5 migration files
services/	Business logic (organization-service)
tests/	Unit (14 files), integration (6 files), isolation (2 files)

2. Performance Deep Dive

Overall Rating: Needs Improvement

Solid foundation with Redis caching and React Query, but several patterns will cause degradation under load.

2.1 Critical Performance Issues

[P1] Duplicated permission checks across 4 API routes (app/api/roles/*.ts)

Each of the four role management routes performs **3 sequential database queries** to check permissions. Should use the cached `resolvePermissions()` from `lib/permissions/resolver.ts` (5-min Redis TTL).

[P2] hooks/usePermission.ts uses raw fetch in useEffect

Three hooks each fire a network request on mount with **zero caching**. If 10 components use these hooks, that's 10 identical in-flight requests. The `features/permissions/api/usePermissions.ts` hook already exists with 30-minute React Query caching.

[P3] Session invalidation on org status change is $O(n)$

The `invalidateOrgSessions()` function does: (1) `findMany` all members, (2) Redis `DEL` per user, (3) `deleteMany` sessions. For an org with 1000+ members, that's **1000+ Redis operations** sequentially.

2.2 Database Performance

[P4] Two separate PrismaClient instances (lib/db.ts + lib/global-db.ts)

Each gets a default pool (max 10 connections). That's **20 of your PgBouncer slots** consumed just for Prisma, leaving less headroom.

[P5] Permission resolution uses include instead of select (lib/permissions/resolver.ts:26-34)

The deeply nested query fetches full model objects when only `permission.key` is needed. Using `select` would significantly reduce data transferred.

[P6] Slug uniqueness loop is $O(n)$ with sequential DB lookups (services/organization-service.ts:62-72)

The slug generation does up to 100 sequential `findUnique` calls. A unique constraint + catch, or a batch `findMany` with candidate slugs would be faster.

2.3 Caching Gaps

[P7] Zero server-side caching on API routes

None of the 16 route files use `revalidate`, `cache: 'force-cache'`, or Next.js cache tags. Every API call hits the auth + DB layer fresh.

***[P8] SSE mock sends random notifications every 15s
(app/api/notifications/stream/route.ts:24)***

Each SSE connection holds an open HTTP request. The mock pattern doesn't scale — use Redis Pub/Sub or a message queue for production.

2.4 Frontend Performance

***[P9] RequirePermission violates React's rules of hooks
(components/auth/RequirePermission.tsx:48-62)***

Hooks are called conditionally based on whether permission is a string or array. This can cause React warnings and unpredictable re-render behavior.

[P10] Admin layout is entirely a client component (app/admin/layout.tsx)

The entire admin sidebar, header, and navigation re-render on every client-side nav because of `useRouter/usePathname`. Split into a server-component shell + client content area.

Top 5 Performance Recommendations (by impact)

1. Migrate `hooks/usePermission.ts` to use React Query — eliminates unbounded network requests
2. Consolidate permission checks in role routes — replace 3 sequential DB queries with `cached resolvePermissions()`
3. Add server-side caching (`revalidate/cache` directives) to GET API routes — reduces DB load significantly
4. Fix `RequirePermission` conditional hooks — prevents React re-render issues
5. Configure Prisma connection pools explicitly — avoid wasting PgBouncer slots

3. Security Deep Dive

Overall Rating: Moderate

Strong architectural thinking (defense-in-depth tenant isolation, CSP nonces, PII encryption), but several critical and high-severity issues must be addressed before production.

Severity Legend:

■ Critical ■ High ■ Medium ■ Low

CRITICAL Findings

[S1] Plaintext credentials in .env — may be in git history

The .env file contains real PostgreSQL and PgBouncer passwords. Even if .gitignore'd, if this file was ever committed, credentials are compromised and **must be rotated immediately**.

[S2] No trustProxy in BetterAuth (lib/auth.ts)

Without trustProxy: true, behind any reverse proxy/load balancer, BetterAuth sees the proxy's IP instead of the real client IP. This breaks session security (IP binding) and audit log forensic integrity.

[S3] Forged x-forwarded-for headers accepted directly (app/api/admin/permissions/route.ts:152, app/api/roles/route.ts:94)

Without trustProxy: true, any client can send a forged x-forwarded-for header to mask their real IP in audit logs. Attackers can obscure their trail.

HIGH Findings

[S4] Member.role is a free-text string (prisma/schema.prisma:136)

Defined as String @default("member") with no foreign key or constraint. A user could potentially set their role to an arbitrary value via client-side manipulation if the backend trusts this field.

[S5] No rate limiting on authentication endpoints (lib/auth.ts)

Email/password login has no brute-force protection. With disableSignUp: true, only pre-created accounts are targetable, but systematic credential testing is still possible.

[S6] requireNonPlatformOrgRoute skips enforcement when no orgId (lib/authz-route.ts:153-156)

Mutation requests (POST/PUT/PATCH/DELETE) to routes wrapped with this middleware execute **without any authorization check** if the caller omits the org context. This is an authorization bypass.

[S7] globalDb has no runtime enforcement (lib/global-db.ts)

The unscoped Prisma client is documented as SUPER ADMIN ONLY but there's no runtime guard. If a developer accidentally imports it in a non-admin route, tenant isolation is silently bypassed.

[S8] No CSRF protection on state-changing API routes (lib/auth.ts)

BetterAuth uses httpOnly cookies, but custom endpoints accepting POST/PUT/PATCH should validate Origin or Referer headers.

MEDIUM Findings

[S9] CSP allows 'unsafe-inline' for styles (middleware.ts:38)

Opens the door to style-based data exfiltration if any user-controlled content can influence CSS.

[S10] Health endpoint exposes version and uptime (app/api/health/route.ts:158-160)

Returns npm_package_version, server uptime, and PgBouncer connection details. Aids attacker reconnaissance. Should be restricted to internal networks.

[S11] BetterAuth session cookie may lack secure/sameSite (lib/auth.ts:29-36)

Not explicitly configured. In production with HTTPS, should be secure: true and sameSite: 'strict'.

[S12] Permission table has read-all RLS policy (prisma/migrations/20260711000000_rls_policies/migration.sql:89-90)

Any authenticated user can enumerate the entire permission catalog, helping attackers map the authorization model.

LOW Findings

[S13] PII_ENCRYPTION_KEY in .env.example looks real (line 23)

A valid 64-character hex string. Developers might copy this into production, resulting in all PII encrypted with a known key.

[S14] User.role creates dual-authorization model (prisma/schema.prisma:69)

Separate from org-scoped RBAC. The interaction between User.role and MemberRole is unclear and could create authorization confusion.

Top 5 Security Recommendations (by severity)

1. Rotate PostgreSQL/PgBouncer passwords — they may be in git history
2. Add trustProxy: true to BetterAuth + validate x-forwarded-for from last trusted proxy only
3. Add rate limiting to email/password login in BetterAuth (emailAndPassword.rateLimit)
4. Fix requireNonPlatformOrgRoute to deny mutations without org context — closes authorization bypass
5. Add Origin/Referer validation to all state-changing API routes — CSRF protection

Summary Scorecard

Architecture	Strong	Defense-in-depth multi-tenancy, feature-sliced design
	Needs Improvement	Duplicated permission checks, raw fetch in hooks
Security	Moderate	No auth rate limiting, no CSRF protection
	Good	3 test categories, factory patterns, isolation tests
Code Quality	Good	TypeScript strict mode, Zod schemas, Pino logging